Projet MapReduce

Exercice 1:

- Question 1: Afficher les dernières lignes du fichier distant.

```
jojo@jojo:~$ hadoop fs -tail myinput/purchases.txt
31      17:59   Norfolk Toys    164.34  MasterCard
2012-12-31      17:59   Chula Vista     Music   380.67  Visa
2012-12-31      17:59   Hialeah Toys    115.21  MasterCard
2012-12-31      17:59   Indianapolis    Men's Clothing  158.28  MasterCard
2012-12-31      17:59   Norfolk Garden  414.09  MasterCard
2012-12-31      17:59   Baltimore       DVDs    467.3   Visa
2012-12-31      17:59   Santa Ana       Video Games     144.73  Visa
2012-12-31      17:59   Gilbert Consumer Electronics    354.66  Discover
2012-12-31      17:59   Memphis Sporting Goods  124.79  Amex
2012-12-31      17:59   Chicago Men's Clothing  386.54  MasterCard
2012-12-31      17:59   Birmingham      CDs     118.04  Cash
2012-12-31      17:59   Las Vegas       Health and Beauty       420.46  Amex
2012-12-31      17:59   Wichita Toys    383.9   Cash
2012-12-31      17:59   Tucson  Pet Supplies    268.39  MasterCard
2012-12-31      17:59   Glendale        Women's Clothing        68.05   Amex
2012-12-31      17:59   Albuquerque     Toys    345.7   MasterCard
2012-12-31      17:59   Rochester       DVDs    399.57  Amex
2012-12-31      17:59   Greensboro      Baby    277.27  Discover
2012-12-31      17:59   Arlington       Women's Clothing        134.95  MasterCard
2012-12-31      17:59   Corpus Christi  DVDs    441.61  Discover
```

- Question 2 : Que permet de faire chaque ligne de ce code ?

1. *for line in sys.stdin* : Cette ligne lit les entrées ligne par ligne à partir de l'entrée standard (stdin). L'objet sys.stdin est comme un objet de fichier que vous pouvez parcourir pour lire les entrées.
2. *data = line.strip().split("\t")* : Cette ligne traite chaque ligne d'entrée. La fonction strip() supprime les espaces blancs en début et en fin de ligne. La fonction split("\t") divise ensuite la ligne en une liste de chaînes, en utilisant le caractère de tabulation comme délimiteur.
3. *if len(data) == 6* : Cette ligne vérifie si la longueur de la liste data est 6. Cela permet de s'assurer que chaque ligne d'entrée a exactement 6 champs, comme prévu.
4. *date, time, store, item, cost, payment = data* : Cette ligne décompose la liste data en six variables distinctes : date, time, store, item, cost et payment. Cela est basé sur l'hypothèse que chaque ligne d'entrée a ces six champs dans cet ordre.
5. *print "{0}\t{1}".format(store, cost)* : Cette ligne imprime les variables store et cost, séparées par un caractère de tabulation. Les {0} et {1} sont des espaces réservés qui sont remplacés par les arguments fournis à la fonction format().

- Question 3 : Tester mapper.

```
jojo@jojo:~$ head -50 purchases.txt | ./mapper.py
San Jose        214.05
Fort Worth      153.57
San Diego       66.08
Pittsburgh      493.51
Omaha   235.63
Stockton        247.18
Austin  379.6
New York        296.8
Corpus Christi  25.38
Fort Worth      213.88
Las Vegas       53.26
Newark  39.75
Austin  469.63
Greensboro      290.82
San Francisco   260.65
Lincoln 136.9
Buffalo 483.82
San Jose        215.82
Boston  418.94
Houston 309.16
Las Vegas       93.39
Virginia Beach  376.11
Riverside       252.88
Tulsa   205.06
Reno    88.25
Chicago 31.08
Fort Wayne      370.55
San Bernardino  170.2
Madison 16.78
Austin  327.75
Portland        108.69
Riverside       15.41
Reno    80.46
Anchorage       298.86
Pittsburgh      475.26
Spokane 3.85
Spokane 287.65
Fresno  466.64
Omaha   255.68
Anchorage       6.38
Aurora  117.81
Philadelphia    351.31
Fremont 222.61
Anchorage       22.36
Norfolk 189.01
Chandler        414.08
Minneapolis     182.05
Honolulu        345.18
Indianapolis    135.96
```

- Question 4 : Expliquer ce code.

1. *salesTotal = 0 et oldKey = None :* Ces lignes initialisent deux variables. salesTotal est utilisé pour accumuler le total des ventes pour une clé donnée (dans ce cas, un nom de magasin), et oldKey est utilisé pour garder une trace de la clé actuellement traitée.
2. *for line in sys.stdin: :* Cette ligne lit les entrées ligne par ligne à partir de l'entrée standard (stdin).
3. *data = line.strip().split("\t") :* Cette ligne traite chaque ligne d'entrée. La fonction strip() supprime les espaces blancs en début et en fin de ligne. La fonction split("\t") divise ensuite la ligne en une liste de chaînes, en utilisant le caractère de tabulation comme délimiteur.

4. *if len(data) != 2: continue :* Cette ligne vérifie si la longueur de la liste data est 2. Si ce n'est pas le cas, elle passe à la ligne suivante.
5. *thiskey, thisSale = data :* Cette ligne décompose la liste data en deux variables : thiskey (la clé actuelle) et thisSale (la vente actuelle).
6. *if oldKey and oldkey != thiskey: :* Cette ligne vérifie si nous avons changé de clé. Si oldKey est défini et différent de thiskey, cela signifie que nous avons terminé de traiter toutes les lignes pour la clé précédente.
7. *print "{0}\t{1}".format(oldKey,salesTotal) :* Cette ligne imprime la clé précédente et le total des ventes pour cette clé.
8. *salesTotal = 0 :* Cette ligne réinitialise salesTotal à zéro pour la nouvelle clé.
9. *oldKey = thisKey :* Cette ligne met à jour oldKey pour qu'elle corresponde à la nouvelle clé actuellement traitée.
10. *salesTotal += float (thisSale) :* Cette ligne ajoute la vente actuelle (convertie en flottant) au total des ventes pour la clé actuelle.
11. *if oldKey != None: print oldkey,"\t", salesTotal :* Cette ligne est exécutée après la fin de la boucle, et elle imprime le total des ventes pour la dernière clé traitée.

- Question 5 : Tester reducer .

```
jojo@jojo:~$ head -50 purchases.txt | ./mapper.py | sort | ./reducer.py
Anchorage       327.6
Aurora  117.81
Austin  1176.98
Boston  418.94
Buffalo 483.82
Chandler        758.17
Chicago 31.08
Corpus Christi  25.38
Fort Wayne      370.55
Fort Worth      367.45
Fremont 222.61
Fresno  466.64
Greensboro      290.82
Honolulu        345.18
Houston 309.16
Indianapolis    135.96
Las Vegas       146.65
Lincoln 136.9
Madison 16.78
Minneapolis     182.05
Newark  39.75
New York        296.8
Norfolk 189.01
Omaha   491.31
Philadelphia    351.31
Pittsburgh      968.77
Portland        108.69
Reno    168.70999999999998
Riverside       268.29
San Bernardino  170.2
San Diego       66.08
San Francisco   260.65
San Jose        429.87
Spokane 291.5
Stockton        247.18
Tulsa   205.06
Virginia Beach  376.11
```

- Question 6 : Exécuter un job hadoop.
- Question 7 : Quelle est la totalité des ventes du magasin de Buffalo ?

```
jojo@jojo:~$ grep "Buffalo" part-00000
Buffalo 10001941.190000044
```

Exercice 2:

- Question 1: Donner la liste des ventes par catégorie de produits.

```
jojo@jojo:~$ hdfs dfs -cat joboutput1/part-00000
Baby    57491808.4399987
Books   57450757.909999445
CDs     57410753.040002026
Cameras 57299046.64000024
Children's Clothing     57624820.93999915
Computers       57315406.31999927
Consumer Electronics    57452374.13000026
Crafts  57418154.499999866
DVDs    57649212.139999844
Garden  57539833.10999964
Health and Beauty       57481589.56000114
Men's Clothing  57621279.04000117
Music   57495489.69999926
Pet Supplies    57197250.240000054
Sporting Goods  57599085.88999977
Toys    57463477.10999993
Video Games     57513165.580000274
Women's Clothing        57434448.96999933
```

- Question 2: Quelle est la valeur des ventes pour la catégorie Toys ?

```
jojo@jojo:~$ grep "Toys" part-00000
Toys    57463477.10999993
```

- Question 3: Et pour la catégorie Consumer Electronics ?

```
jojo@jojo:~$ grep "Consumer Electronics" part-00000
Consumer Electronics    57452374.13000026
```

- Question 4: Donner le montant de la vente le plus élevé pour chaque magasin.

```
jojo@jojo:~$ cat part-00000
Albuquerque     499.98
Anaheim 499.98
Anchorage       499.99
Arlington       499.95
Atlanta 499.96
Aurora  499.97
Austin  499.97
Bakersfield     499.97
Baltimore       499.99
Baton Rouge     499.98
Birmingham      499.99
Boise   499.98
Boston  499.99
Buffalo 499.99
Chandler        499.98
Charlotte       499.98
Chesapeake      499.98
Chicago 499.99
Chula Vista     499.99
Cincinnati      499.98
Cleveland       499.98
Colorado Springs        499.99
Columbus        499.98
Corpus Christi  499.96
Dallas  499.99
Denver  499.97
Detroit 499.99
Durham  499.96
El Paso 499.98
Fort Wayne      499.96
Fort Worth      499.98
Fremont 499.99
```

- Question 5: Quelle cette valeur pour les magasins suivants : Reno/Toledo/Chandler.

```
jojo@jojo:~$ grep -E "Reno|Toledo|Chandler" part-00000
Chandler        499.98
Reno    499.99
Toledo  499.98
```

- Question 6: Quel est le nombre total des ventes et la valeur totale des ventes de tous magasins confondus?

```
jojo@jojo:~$ cat part-00000
Total Sales: 1034457953.2599697
Total Count: 4138476
```